training the policy network, we need to vary the parameters with respect to a fixed input to find a set of parameters that optimizes our objective (i.e., minimizes a loss or maximizes a utility function), which is the function $\pi_s(\theta)$.

Definition

The probability of an action, given the parameters of the policy network, is denoted $\pi_s(a|\theta)$. This makes it clear that the probability of an action, a, explicitly depends on the parameterization of the policy network. In general, we denote a *conditional probability* as $P(x \mid y)$, read "the probability distribution over x given y." This means we have some function that takes a parameter y and returns a probability distribution over some other parameter x.

In order to reinforce action 3, we want to modify our policy network parameters θ such that we increase $\pi_s(a_3|\theta)$. Our objective function merely needs to maximize $\pi_s(a_3|\theta)$ where a_3 is action 3 in our example. Before training, $\pi_s(a_3|\theta) = 0.25$, but we want to modify θ such that $\pi_s(a_3|\theta) > 0.25$. Because all of our probabilities must sum to 1, maximizing $\pi_s(a_3|\theta)$ will minimize the other action probabilities. And remember, we prefer to set things up so that we're minimizing an objective function instead of maximizing, since it plays nicely with PyTorch's built-in optimizers—we should instead tell PyTorch to minimize $1 - \pi_s(a|\theta)$. This loss function approaches 0 as $\pi_s(a|\theta)$ nears 1, so we are encouraging the gradients to maximize $\pi_s(a|\theta)$ for the action we took. We will subsequently drop the subscript a_3, as it should be clear from the context which action we're referring to.

4.2.3. Log probability

Mathematically, what we've described is correct. But due to computation imprecisions we need to make adjustments to this formula to stabilize the training. One problem is that probabilities are bounded by 0 and 1 by